Smart Parking System — Research Document

Stage 3: research, simulation and documentation. Sending every arriving car to the nearest free stall with A*: an ESP32 prototype on a 1:100 model and a simulation study of what it saves.

Course	A Project in Internet of Things (IoT) — Technion, Faculty of Computer Science (ICST)
Authors	Aseel Kurdy, Nayef Habiballah, Osama Abdulhalim
Instructors	Itai Dabran, Ido Ram, Diana Cohen
Date	25/10/2026
Repository	github.com/AseelKurdy/smart-parking-lot (firmware, tests, simulation, documents)
Demo video	release v1.0 — SmartParking_demo.mp4 (3 min)

1. General description

Drivers who look for a parking space on their own drive further, burn more fuel and lose time. We built a smart parking system that **assigns every arriving car the nearest free stall**, tells the driver where to go, and checks whether the car parked where it was sent. The prototype is a two-floor, 14-stall parking lot built at a scale of 1:100 and run by a single ESP32 board. An ultrasonic sensor across the entrance counts arriving cars, a second ultrasonic sensor above one stall confirms real parking, and an LED tells the driver whether they are in the right stall. The lot is stored as a road network with the lengths measured on the model, and the A* algorithm finds the shortest driving route to every stall. A reserved stall is held for 60 seconds. The board creates its own Wi-Fi network and serves a live 2D/3D map of the lot to any phone or laptop.

To find out whether the idea pays off in a real garage, the same assignment rule was evaluated in a simulation of a 216-stall, three-floor garage with arrival and parking-time data from published studies. At 85% occupancy, assigning the stall saves **85 m of driving per car (31 s, 8.5 mL of fuel, 20 g of CO_2)** compared with a driver who searches aisle by aisle and notices every free stall; over a year this is about 4,700 driver-hours, 4,700 litres of fuel, 11 tonnes of CO_2 and ₪166,000. The firmware itself was verified with 54 automated tests on a PC simulator of the board.

2. Introduction

The need. Searching for parking is a measurable waste. A review of 16 studies found that drivers cruised for parking for 8.1 minutes on average and that cruising made up about 30% of traffic in the congested downtown areas studied (Shoup, 2006). Inside off-street facilities the search is shorter but still real: drivers reported an average search of 50 s off-street against 30 s on-street in a survey of 11,425 trips (van Ommeren, Wentink & Rietveld, 2012), and a GPS study measured a mean of 1.5 minutes of cruising (Saki & Hagen, 2024). Search effort also rises sharply with occupancy: it becomes noticeable above about 85% occupancy and exceeds a minute above 92–93% (Levy, Martens & Benenson, 2013).

Solutions today. Published systems attack the problem in two ways. *Guidance and information systems* tell drivers where free spaces are: zone-level availability information cut search time in a parking facility by 16.2% in simulation (Caicedo, 2009), a smart parking system saved 11 s of search per car in a university lot (Surpris, Liu & Vincenzi, 2014), and San Francisco's sensor-based SFpark pilot reported that the average search time in its pilot areas fell by 43% against control areas (SFMTA, 2014). *Assignment and reservation systems* go further and give each driver a specific space: allocating and reserving spaces roughly halved the time to park in simulation (Geng & Cassandras, 2011). Surveys of the field describe the same split between information collection, system deployment and service dissemination (Lin, Rivano & Le Mouël, 2017).

What we propose. A self-contained system in the second category, small enough to run on one microcontroller: the lot itself decides where each car should go, using the real driving distances of the lot, holds the stall for the driver for a limited time, detects drivers who park elsewhere and immediately re-balances the reservations, and shows everything live on a web page served by the board — with no server, cloud or internet connection. Figure 1 shows the model and the live view.

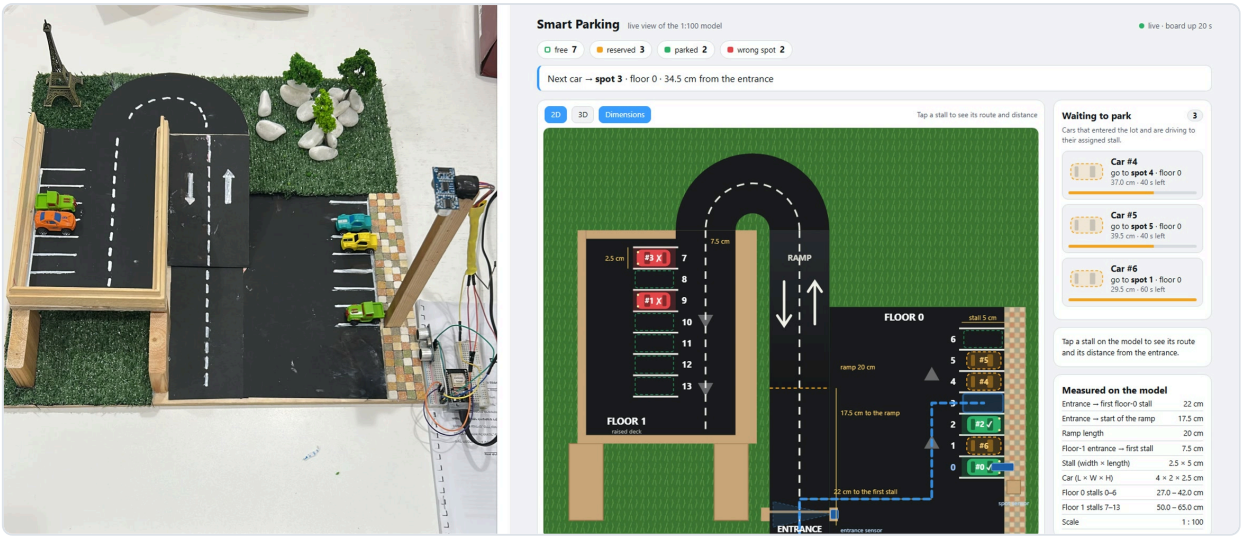


Figure 1. Left: the 1:100 model — floor 1 on the raised deck (left), the ramp with its U-turn, floor 0 (right), the entrance sensor on its post and the ESP32 board. Right: the live web view served by the board.

3. Related work

Smart parking as a field. Smart parking grew from simple car-park counting and guidance signs into networked systems that sense individual spaces and share the data with drivers. Idris et al. (2009) reviewed the early generation of such systems and their technologies, from parking guidance and information (PGI) systems to reservation and e-parking concepts, with a focus on how vehicles are detected. Lin, Rivano & Le Mouël (2017) surveyed the literature of 2000–2016 and organised it into three themes: *information collection* (sensors and crowd-sourcing), *system deployment* (models, allocation, pricing and reservation) and *service dissemination* (how results reach drivers). Al-Turjman & Malekloo (2019) reviewed smart parking in IoT-enabled cities, classifying systems by their hardware and software design factors and highlighting data reliability, security and privacy as open issues. Our project combines all three of Lin et al.'s themes in one device: it collects occupancy data with sensors, runs an allocation model, and disseminates the result through a web page.

Allocation and reservation. Most deployed systems only report free spaces and leave the choice to the driver. Geng & Cassandras (2011) formulated parking as an optimal resource-allocation problem with reservations and showed in simulation that assigning spaces roughly halves the time to park. Caicedo (2009) showed that even coarse, zone-level availability information shortens the search. Real deployments such as SFpark (SFMTA, 2014) combined space-level sensors with demand-responsive pricing and reported less searching and less circling traffic. Our system follows the assignment approach: the nearest stall by actual driving distance, held for a limited time.

Sensing technologies. Vehicle-detection sensors fall into *intrusive* sensors installed in or under the road surface (inductive loops, magnetometers) and *non-intrusive* sensors mounted above or beside the space (ultrasonic, infrared, microwave radar, cameras) (Idris et al., 2009). In-ground sensors are robust but need roadworks; cameras cover many spaces at once but need image processing, lighting and raise privacy questions; ultrasonic sensors are cheap, simple and well suited to indoor garages, where they are commonly mounted above each space. For a 1:100 model with a microcontroller, an ultrasonic range sensor is the natural choice: it measures a distance directly, needs no calibration other than thresholds, and works in a closed structure.

The sensor we used: HC-SR04. The HC-SR04 is an ultrasonic ranging module powered from 5 V (about 15 mA). A 10 µs pulse on its *Trig* input makes it send a burst of eight 40 kHz ultrasonic pulses; it then raises its

Echo output for as long as the sound takes to travel to the nearest object and back. The distance is half the round-trip time multiplied by the speed of sound (about 343 m/s at 20 °C), which the datasheet gives as *distance [cm] = echo time [µs] / 58*. The module measures 2–400 cm with a resolution of about 3 mm within a measuring angle of about 15°, and the datasheet recommends a measurement cycle of at least 60 ms so that an echo cannot be confused with the next burst (ElecFreaks, n.d.). These figures shaped our design: the two sensors are fired 60 ms apart; readings below the 2 cm minimum are treated as electrical noise; and the 3 mm resolution sets the limit on how close our thresholds can be (Section 4.3).

4. Project description

4.1 What we built

The **model** is a 1:100 parking lot on a wooden base (Figure 1): black card roads with painted lane markings and stalls, artificial grass, a raised wooden deck for floor 1, and a ramp with a U-turn between the floors. Every length the software uses was measured on the model: a stall is 2.5 × 5 cm (a standard 2.5 × 5 m bay), a car 4 × 2 × 2.5 cm, the floor-0 lane starts 22 cm from the entrance, the ramp starts 17.5 cm from the entrance and is 20 cm long, and the first floor-1 stall is 7.5 cm past the ramp.

The **electronics** are an ESP32 development board (dual-core 240 MHz microcontroller with built-in Wi-Fi; Espressif Systems, 2023), two HC-SR04 sensors and the board's built-in LED. The firmware is written in C++ with the Arduino core for ESP32, version 3.0.1 (Espressif Systems, n.d.).

Function	Device	Trig	Echo
Entrance detection (sideways across the road)	HC-SR04	GPIO 27	GPIO 14
Stall 0 detection (from above)	HC-SR04	GPIO 26	GPIO 25
Correct / wrong parking indicator	built-in LED	—	GPIO 2

One stall (stall 0) has a physical sensor; the other 13 stalls report parking through a serial command, which demonstrates the full logic with one sensor per stall where a real lot would have one sensor in every stall.

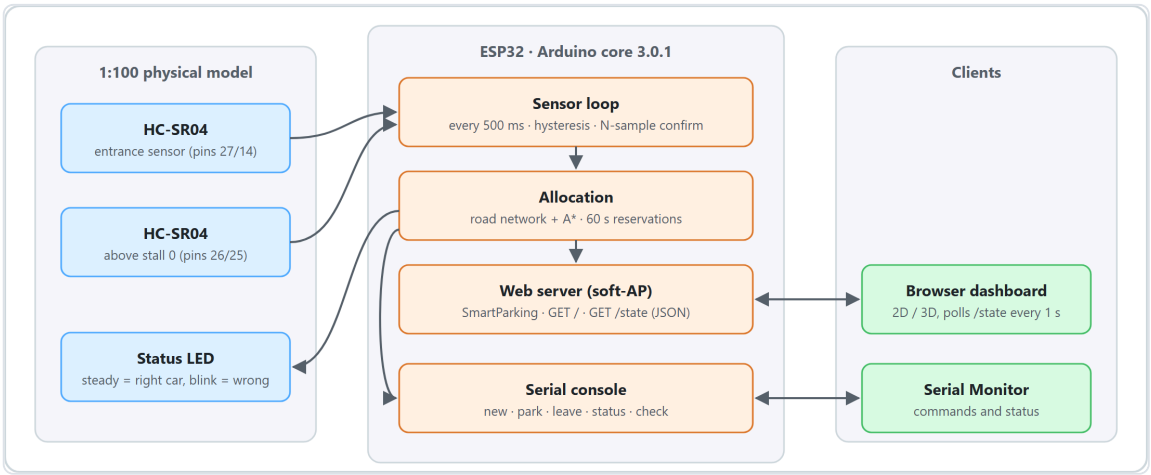


Figure 2. System architecture: sensors and LED on the model, the four parts of the firmware on the ESP32, and the two clients.

4.2 Communication

- **Wi-Fi.** The ESP32 runs as a Wi-Fi access point named *SmartParking*, so no router or internet is needed. It serves an HTTP server on port 80 with two endpoints: `GET /` returns the web page (HTML, CSS and

JavaScript stored in the board's flash memory) and `GET /state` returns the full state of the lot as JSON — every stall's state, the car in it, the waiting cars with their remaining reservation time, and the measured geometry.

- **Live view.** The page in the phone's or laptop's browser requests `/state` once a second and redraws the lot in 2D or 3D: free, reserved (with countdown), parked correctly, wrong stall and blocked stalls, the route the next car will take, and a waiting list.
- **Serial.** Over USB (9600 baud) the board prints every event and accepts commands: `new`, `park <stall> <car>`, `leave <car>`, `status` and `check [seconds]` (raw sensor readings for calibration).

4.3 Algorithms

Road network and A*. The lot is a graph built at start-up from the measured lengths (Figure 3): the entrance, the start of the ramp, the floor-1 entrance, a lane point in front of each stall (2.5 cm apart — one stall width) and the 14 stalls, each hanging off its lane point by its length (5 cm). A stall can only be a destination, never driven through. The driving distance to every stall is found with A* (Hart, Nilsson & Raphael, 1968), a best-first search that expands the node with the smallest $f = g + h$, where g is the distance driven so far and h an estimate of the remaining distance. Our heuristic is the stall length (every route ends by driving into the stall) plus, on the goal's own lane, the remaining distance along the lane. It never overestimates the true distance, so A* returns the shortest path, as Dijkstra's algorithm would (Dijkstra, 1959), while expanding fewer nodes. Because the lot is fixed, the 14 distances are computed once at boot (27.0–42.0 cm on floor 0, 50.0–65.0 cm on floor 1), and assigning a stall at run time is a lookup.

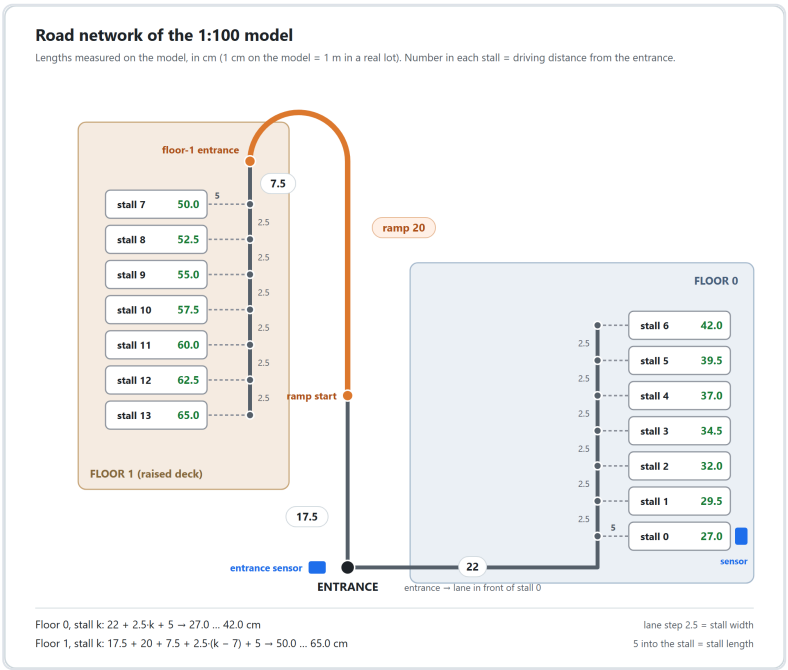


Figure 3. The road network drawn in the model's real layout, with the measured lengths and every stall's driving distance from the entrance.

Assignment rules. A new car gets the nearest stall that is free, not reserved for another car and not blocked, and the stall is reserved for 60 seconds. If the car parks in its stall it is *correct*. If it parks in another free stall, its own reservation is released at once and the stall is marked *wrong*. If it takes a stall reserved for another car, that car is immediately re-routed to the nearest free stall. A reservation that expires returns the stall to the pool. An object detected on stall 0 with no car on its way marks the stall *blocked* so nobody is sent there.

Sensor processing. Both sensors are read every 500 ms. The entrance sensor reads about 30 cm (the wall opposite) when the road is empty and about 22 cm with a car; a car is counted below 25 cm after two consecutive readings, and only once until the road reads empty twice — a car that stops in front of the sensor is not counted again. The stall-0 sensor reads 25–26 cm when the stall is empty and 22.5–24.5 cm with a car (depending on its height, 2.5 or 1.5 cm). It uses hysteresis — a car arrives below 24.7 cm and leaves above 24.9 cm — and a change of state needs four consecutive valid readings (2 s), so a reading near a threshold cannot make the state flicker. Readings with no echo or below the sensor's 2 cm minimum are invalid and restart the count; this removed phantom cars caused by electrical noise on the Echo line, which our automated tests found on the real board.

5. Simulation description

What we checked and how. The question was how much driving, time, fuel, CO_2 and money the assignment rule saves compared with drivers who search on their own. We wrote a discrete-event simulation in Python that follows individual cars through a parking lot (Appendices A and B). Cars arrive as a Poisson process and stay for a gamma-distributed time with a mean of 88 minutes, both taken from a study of a mall garage (Huang et al., 2020); the arrival rate for each target occupancy is found from the Erlang loss model of a lot with c stalls. Three strategies are applied to exactly the same cars (common random numbers): **assigned** — the firmware's rule, the nearest free stall by A*; **searching alone** — a driver who searches aisle by aisle, as people do, noticing each free stall with probability p (p = 1, the most favourable case for the driver, and p = 0.5); and **random search**, as an upper bound. Two lots were simulated: the prototype's 14-stall lot and the main scenario, a 216-stall, three-floor garage with three two-way aisles per floor, built to the dimensions of the Israeli parking design guidelines (Israel Ministry of Transport, 2000). Each setting was run at eight occupancy levels from 50% to 98%, with 20 replications of 3,000 measured cars after a warm-up. Distances are converted to time, fuel, CO_2 and money with published constants (driving speed 10 km/h, 1 L/h of fuel at crawl speed, 2.35 kg of CO_2 per litre, the October 2026 fuel price and the official value of travel time; each with its source in `simulation/params.py`). Eleven automated tests check the simulation, among them that A* agrees with an exhaustive Dijkstra search for every stall and that the simulated occupancy matches the Erlang formula.

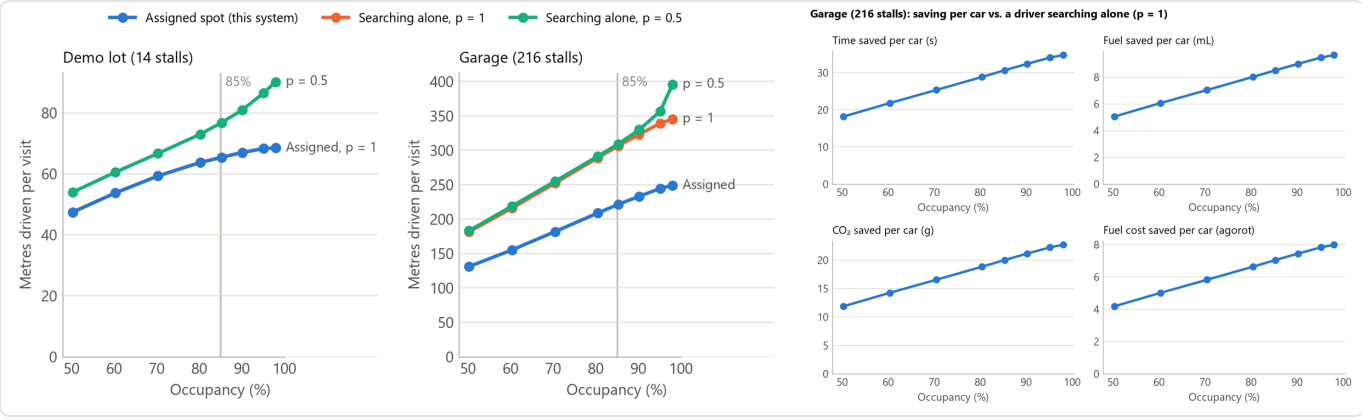


Figure 4. Left: metres driven inside the lot per visit against occupancy, for both lots. Right: savings per car in the garage compared with a driver searching alone (p = 1). Shaded bands are 95% confidence intervals.

Results. In the garage, the assigned driver always drives less than the driver searching alone, and the gap grows as the garage fills (Figure 4, left): at 85% occupancy the assigned driver drives 222 m per visit and the searching driver 307 m. Being assigned cuts the driving to the stall by 43–44% at every occupancy level. The

saving per car grows from 18 s at 50% occupancy to 35 s at 98%, and to 53 s at 98% when the driver misses half of the free stalls (Figure 4, right).

Garage (216 stalls), 85% occupancy, assigned vs. searching alone (p = 1)	**Per car**	**Per year**
Driving saved	85.2 ± 1.1 m	46,675 km
Time saved	30.7 s	4,667 h
Fuel saved	8.5 mL	4,667 L
CO_2 avoided	20.0 g	11.0 t
Money value (fuel + drivers' time)	₪0.30	₪166,069

How the project helps. Per car the saving is small, but a garage of this size serves about 548,000 cars a year, so seconds and millilitres add up to thousands of hours and litres. The result is a lower bound: the searching driver is modelled favourably (an efficient route, every free stall noticed, no circling), and it agrees with published figures without being tuned to them — 31 s of extra search is below the 50 s reported for off-street parking (van Ommeren et al., 2012), and the 43–44% reduction lies inside the 16–50% range reported for information and reservation systems (Caicedo, 2009; Geng & Cassandras, 2011). The benefit comes from routing drivers past full aisles and floors: in the prototype's small single-lane lot, a driver who notices every free stall already finds the nearest one, so the system helps there only when drivers miss stalls (11.3 m per car at p = 0.5) or when the lot is full. The largest uncertainty is the driving speed inside the garage: between 20 and 5 km/h, the yearly saving ranges from ₪83,000 to ₪332,000.

The firmware was checked in simulation too. The unmodified firmware was compiled on a PC with simulated sensors, serial port, LED and web server, and an automated suite of 54 tests (parser, allocation rules, A* distances against an independent formula, timeouts, sensor thresholds, web data) passed in full; a model-based test compared the firmware with an independent model of the rules after each of 161,040 random operations with no difference. The behaviour on the real board was demonstrated in the recorded video.

6. Conclusions

- A working ESP32 prototype on a measured 1:100 model counts arriving cars, assigns the nearest free stall by real driving distance using A*, holds it for 60 seconds, detects correct and wrong parking with a sensor and an LED, keeps reservations consistent when drivers do not follow them, and shows the lot live on a web page served by the board itself.
- In a simulated 216-stall garage at 85% occupancy, assigning the stall saves 85 m, 31 s, 8.5 mL of fuel and 20 g of CO_2 per car — about ₪166,000 a year in fuel and drivers' time — and the saving grows with occupancy. Because the searching driver is modelled favourably, this is a lower bound, and it agrees with published figures.
- The benefit is largest where the order in which a driver searches differs from the shortest path: garages with several aisles and floors. In a small single-lane lot it is limited to drivers who miss free stalls and to cars that would otherwise search a full lot.
- Next steps: a sensor in every stall instead of one physical stall, sturdier sensor wiring (separate Trig/Echo wires, a decoupling capacitor and a voltage divider on the 5 V Echo line), and simulating daily arrival peaks and drivers who ignore their assignment.

References

- Al-Turjman, F., & Malekloo, A. (2019). Smart parking in IoT-enabled cities: A survey. *Sustainable Cities and Society, 49*, 101608. https://doi.org/10.1016/j.scs.2019.101608
- Caicedo, F. (2009). The use of space availability information in "PARC" systems to reduce search times in parking facilities. *Transportation Research Part C, 17*(1), 56–68. https://doi.org/10.1016/j.trc.2008.07.001
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*, 269–271. https://doi.org/10.1007/BF01386390
- ElecFreaks. (n.d.). *Ultrasonic ranging module HC-SR04* [Datasheet]. https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf
- Espressif Systems. (2023). *ESP32 series datasheet* (v4.x). https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Espressif Systems. (n.d.). *Arduino-ESP32 documentation* (Wi-Fi and WebServer libraries). https://docs.espressif.com/projects/arduino-esp32/en/latest/
- Geng, Y., & Cassandras, C. G. (2011). A new "smart parking" system based on optimal resource allocation and reservations. *Proc. 14th IEEE ITSC*. https://people.bu.edu/cgc/Published/ITSC11.pdf
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136
- Huang, X., Long, X., Wang, J., & He, L. (2020). Research on parking sharing strategies considering user overtime parking. *PLOS ONE, 15*(6), e0233772. https://doi.org/10.1371/journal.pone.0233772
- Idris, M. Y. I., Leng, Y. Y., Tamil, E. M., Noor, N. M., & Razak, Z. (2009). Car park system: A review of smart parking system and its technology. *Information Technology Journal, 8*(2), 101–113. https://doi.org/10.3923/itj.2009.101.113
- Israel Ministry of Transport. (2000). *Parking planning guidelines, Chapter D: Planning parking facilities*. https://www.gov.il/BlobFolder/policy/parking_planning_guidelines/he/parking.pdf
- Levy, N., Martens, K., & Benenson, I. (2013). Exploring cruising using agent-based and analytical models of parking. *Transportmetrica A, 9*(9), 773–797. https://doi.org/10.1080/18128602.2012.664575
- Lin, T., Rivano, H., & Le Mouël, F. (2017). A survey of smart parking solutions. *IEEE Transactions on Intelligent Transportation Systems, 18*(12), 3229–3253. https://doi.org/10.1109/TITS.2017.2685143
- Saki, S., & Hagen, T. (2024). Cruising for parking again. *Transportation Research Part A, 183*. https://doi.org/10.1016/j.tra.2024.104045
- SFMTA — San Francisco Municipal Transportation Agency. (2014). *SFpark pilot project evaluation*. https://www.sfmta.com/getting-around/drive-park/demand-responsive-pricing/sfpark-evaluation
- Shoup, D. C. (2006). Cruising for parking. *Transport Policy, 13*(6), 479–486. https://doi.org/10.1016/j.tranpol.2006.05.005
- Surpris, G., Liu, D., & Vincenzi, D. (2014). How much can a smart parking system save you? *Ergonomics in Design, 22*. https://doi.org/10.1177/1064804614526202
- van Ommeren, J. N., Wentink, D., & Rietveld, P. (2012). Empirical evidence on cruising for parking. *Transportation Research Part A, 46*(1), 123–130. https://doi.org/10.1016/j.tra.2011.09.011

The sources of the simulation's cost constants (fuel use, CO_2, fuel price, value of travel time) are recorded with each constant in `simulation/params.py`.

Appendix A — Simulation user guide

A.1 What the simulation does

The simulation estimates how much driving, time, fuel, CO_2 and money is saved when every arriving car is sent to the nearest free stall (this project's rule), compared with drivers who search on their own. It runs two lots — the prototype's 14-stall lot and a 216-stall, three-floor garage — at eight occupancy levels, and writes the results as tables and figures.

A.2 Requirements and installation

- Python 3.8 or newer (tested with Python 3.12.10).
- Three libraries: `numpy`, `pandas` and `matplotlib` (tested with numpy 2.5.4, pandas 3.1.0, matplotlib 3.11.2).
- The code: folder `simulation/` of the repository github.com/AseelKurdy/smart-parking-lot (*Code* → *Download ZIP*, or `git clone`).

```
pip install numpy pandas matplotlib
```

A.3 Running it

All commands run from inside the `simulation/` folder.

Command	What it does	Time
`python test_sim.py`	Runs the 11 automated checks (A* vs. Dijkstra, the Erlang formula, reproducibility …). All must print `OK`.	≈ 6 s
`python run_all.py`	Runs every experiment and writes `results/*.csv` and `figures/*.png` (existing figures are replaced).	≈ 3 min
`python run_all.py --figures`	Redraws the figures from the saved results without running the simulation again.	seconds

The same random seeds are used every time, so a run reproduces the published results exactly.

A.4 Outputs

File	Contents
`results/headline.csv`	The main result: per lot and baseline at 85% occupancy — metres, seconds, fuel, CO_2 and money saved per car and per year.
`results/occupancy_sweep.csv`	Mean and 95% confidence interval of every measure, per lot, *p* and occupancy level.
`results/occupancy_sweep_replications.csv`	The same, one row per replication (raw data).
`results/parameters.csv`	Every constant used, with its source, page and sensitivity range.
`results/sensitivity.csv`	The yearly saving when each uncertain constant is set to its low and high value.
`results/validation.csv`	The model's search time and driving reduction next to the published benchmark figures.
`figures/fig1_driving_vs_occupancy.png`	Metres driven per visit against occupancy (Figure 4, left).
`figures/fig2_savings_vs_occupancy.png`	Savings per car against occupancy (Figure 4, right).
`figures/fig3_sensitivity.png`	Sensitivity of the yearly saving to each uncertain constant.

A.5 Changing the scenario

- **A constant** (driving speed, fuel use, garage size, parking duration …): change its `value` in `params.py`. Each constant keeps its source next to it; uncertain ones also have a `sweep` range used by the sensitivity analysis.
- **The occupancy levels, replications or cars per replication:** `OCCUPANCIES`, `REPS` and `CARS` at the top of `run_all.py`.
- After a change, run `python test_sim.py` and then `python run_all.py`.

Appendix B — Simulation programmer guide

B.1 Structure

File	Role
`params.py`	Every constant (`PARAMS`), each a `Param(value, unit, source, where, note, verified, sweep)` ; `derived()` computes the step costs and the ramp length from them; `BENCHMARKS` are published figures used only to check the results; `REFERENCES` the sources. Single source of truth: nothing else hard-codes a number.
`lot.py`	Lot geometry and path finding. `Lot` stores a lot as text grids (`.` lane, `P` stall, `E` entrance, `R` ramp, `#` blocked) with step costs; `a_star()` is the firmware's A* rule; `dijkstra()` checks it; `demo_lot()` builds the 14-stall lot and `garage()` the multi-storey garage with its search route.
`sim.py`	The simulation. `LotModel` precomputes, per lot, each stall's A* distance, the order in which a searching driver passes the stalls, and the distances between stalls. The strategies `assigned()` , `cruising()` and `random_search()` take the current occupancy and return the chosen stall and the metres driven. `simulate()` runs one replication; `erlang_b()` and `arrival_rate_for()` give the arrival rate for a target occupancy; `to_costs()` converts metres to seconds, fuel, CO_2 and money.
`run_all.py`	The experiments: `build_models()` , `run()` (all replications of one setting), `aggregate()` (means and 95% confidence intervals), the yearly totals, the sensitivity analysis and the three figures; `main()` writes everything to `results/` and `figures/` .
`test_sim.py`	Eleven `unittest` checks of the geometry, A*, the search route, the Erlang formula, the occupancy calibration and reproducibility.

B.2 How a replication works

1. **Demand.** `simulate()` draws, from one random generator, each car's inter-arrival time (exponential, rate λ cars/min) and stay (gamma, mean 88 min, shape 1.8). Every strategy sees exactly these cars (common random numbers).
2. **Events.** Before each arrival, departures due by then are processed from a heap of `(departure time, car)` . Each strategy keeps its own occupancy array, so the lots evolve independently.
3. **Choosing a stall.** For each strategy, the arriving car is placed by `assigned()` , `cruising()` or `random_search()` ; a car that finds the lot full leaves (the full-lot driving is recorded). The metres to the stall and back to the exit (the shortest path) are stored per car.
4. **Warm-up.** The first `max(500, 4·λ·mean stay)` cars only fill the lot to steady state and are not measured; then 3,000 cars are measured.
5. **Summary.** `summarize()` averages the metres per parked car and the share of cars turned away; `run_all.py` aggregates 20 replications per setting.

B.3 The arrival rate for a target occupancy

With offered load a = λ·E[S] and c stalls, the Erlang-B probability $B(c, a)$ is the share of cars turned away, and the mean occupancy is $a \cdot (1 - B)/c$. `arrival_rate_for()` solves this for λ by bisection, so each experiment runs at the occupancy it is labelled with (checked by a test to within 0.02).

B.4 Extending the simulation

- **A new strategy:** write a function `(occ, m, rng, p) -> (stall or -1, metres)` in `sim.py` and add it to `STRATEGIES`; it is then run on the same cars as the others.
- **A new lot:** build a `Lot` (grids, entrance, ramps, stall list and search route) in `lot.py`, add it to `build_models()` in `run_all.py`; the tests in `test_sim.py` show how to check its distances against Dijkstra.
- **A new constant:** add a `Param` to `PARAMS` with its source; give it a `sweep` range to include it in the sensitivity analysis.

Appendices C and D — Stage-2 documents

The stage-2 documents are submitted as separate files with this report:

- **Appendix C — Programmer manual** (`programmer_manual.pdf`, English): build and toolchain, firmware structure, sensor calibration constants, the road network and A*, the JSON interface and the web page, the automated tests, and a function index.
- **Appendix D — User manual** (`user_manual.pdf`, Hebrew): installing and running the system, the commands, the web page, calibration values and troubleshooting.

Also submitted: the project documentation (`project_documentation.pdf`), the demo video, the simulation graphs and screenshots of the running system. The source code, tests and simulation are in the repository: github.com/AseelKurdy/smart-parking-lot.